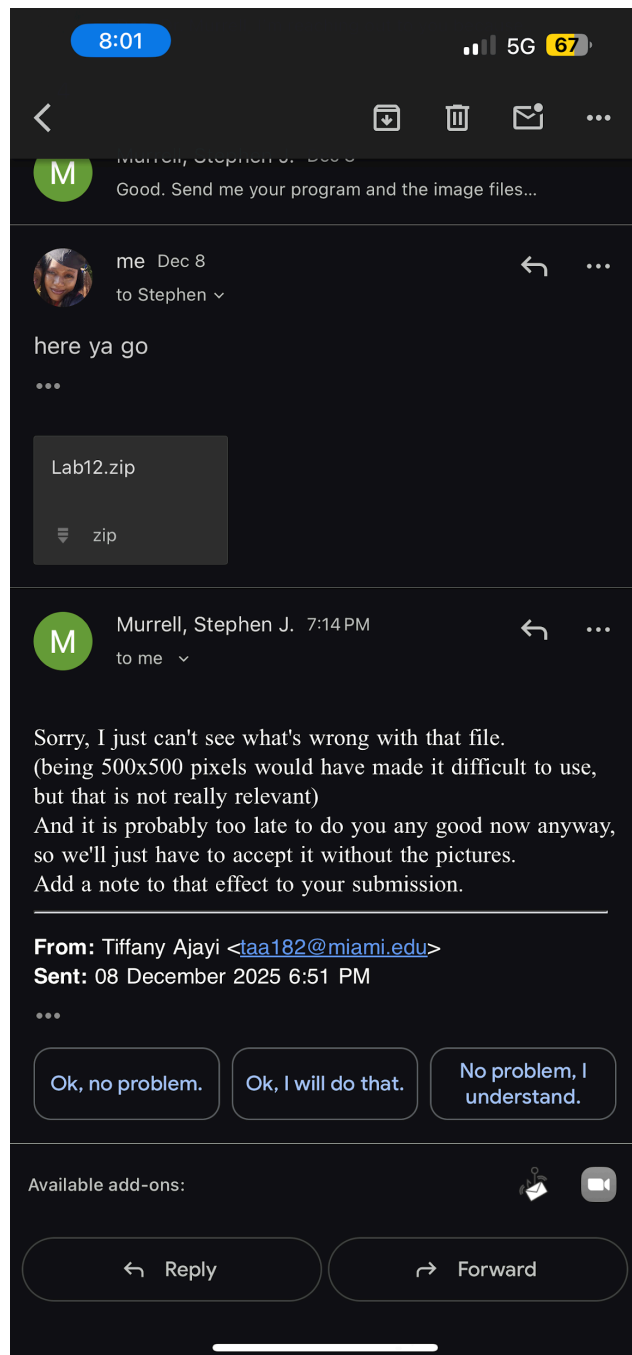


DISCLAIMER!!!!!!

Just a heads up, unfortunately the image graphics don't display in my program. For weeks I tried EVERYTHING I could think of and even went over it with Dr. Murrell, but we just couldn't figure out why it wasn't working. He told me it was okay to submit without the graphics and to leave a note for you. I've attached his email as proof. Thanks for understanding.



7. Better Graphics

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

// THE PART THATS GIVING ME HELL!!!!!!!!!!!!

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else
    {
        set_pen_color(color::white);
    }
}
```

```

fill_rectangle(x, y, s, s);
square_outline(x, y, s);

if (which == 1)
{
    image* treasure_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual
Studio 2010/Projects/Lab12/Lab12/treasure.bmp");
    draw_image(treasure_icon, x, y);
}
else if (which == 2)
{
    image* robot_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual Studio
2010/Projects/Lab12/Lab12/robot.bmp");
    draw_image(robot_icon, x, y);
}
}

```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'a' key to put the robot in automatic mode." <<
endl << endl << "Try to make it to the treasure :) (green sqaure)" << endl;

    char maze[size][size];
}

```

```

int rows = 0;
int columns = 0;
int start_row = -1, start_column = -1;
int treasure_row = -1, treasure_column = -1;

ifstream fin("maze2a.txt");

if (!fin)
{
    cout << "Can't open file :(";
}

string line;

while (getline(fin, line))
{
    columns = (int)line.length();

    for (int c = 0; c < columns; c++)
    {
        maze[rows][c] = line[c];

        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}

fin.close();

make_window(columns * cell, rows * cell);
set_pen_width(1);

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)

```

```

{
    int x = c * cell;
    int y = r * cell;

    if (maze[r][c] == 'X')
    {
        walls(x, y, cell);
    }
    else if (maze[r][c] == '+')
    {
        filled_square(x, y, cell, 0);
    }
    else if (maze[r][c] == '$')
    {
        filled_square(x, y, cell, 1);
    }
    else
    {
        moving_space(x, y, cell);
    }
}
}

filled_square(start_column * cell, start_row * cell, cell, 2);

int robot_row = start_row;
int robot_column = start_column;

bool automatic = false;
const int moves = 11;

while (true)
{
    char c = 0;

    if (!automatic)
    {
        c = wait_for_key_typed();
    }
    else
    {
        for (int n = 0; n < moves; ++n)
        {
            int r = rand() % 4;

```

```

if (r == 0)
    {
        c = -91;
    }
if (r == 1)
    {
        c = -89;
    }
if (r == 2)
    {
        c = -90;
    }
if (r == 3)
    {
        c = -88;
    }

int new_row = robot_row;
int new_column = robot_column;

if (c == -91)
    {
        new_column--;
    }
if (c == -89)
    {
        new_column++;
    }
if (c == -90)
    {
        new_row--;
    }
if (c == -88)
    {
        new_row++;
    }
if (maze[new_row][new_column] == 'X')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 2);
}
else
{
    if (maze[robot_row][robot_column] == '+')

```

```

        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
    else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
    else
        {
            moving_space(robot_column * cell, robot_row * cell, cell);
        }

    robot_row = new_row;
    robot_column = new_column;

    filled_square(robot_column * cell, robot_row * cell, cell, 2);
}

wait(0.15);
}

char k = wait_for_key_typed();

if (k == 'q')
    {
        break;
    }
if (k == 'a')
{
    automatic = false;
    continue;
}
else
{
    automatic = true;
    continue;
}
}

if (c == 'q')

```

```

        {
            break;
        }
    if (c == 'a')
    {
        automatic = true;
        continue;
    }

    int new_row = robot_row;
    int new_column = robot_column;

    if (c == -91)
    {
        new_column--;
    }
    if (c == -89)
    {
        new_column++;
    }
    if (c == -90)
    {
        new_row--;
    }
    if (c == -88)
    {
        new_row++;
    }
    if (maze[new_row][new_column] == 'X')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
        continue;
    }
    if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
    else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
    else
    {
        moving_space(robot_column * cell, robot_row * cell, cell);
    }

```

```

    }

    robot_row = new_row;
    robot_column = new_column;

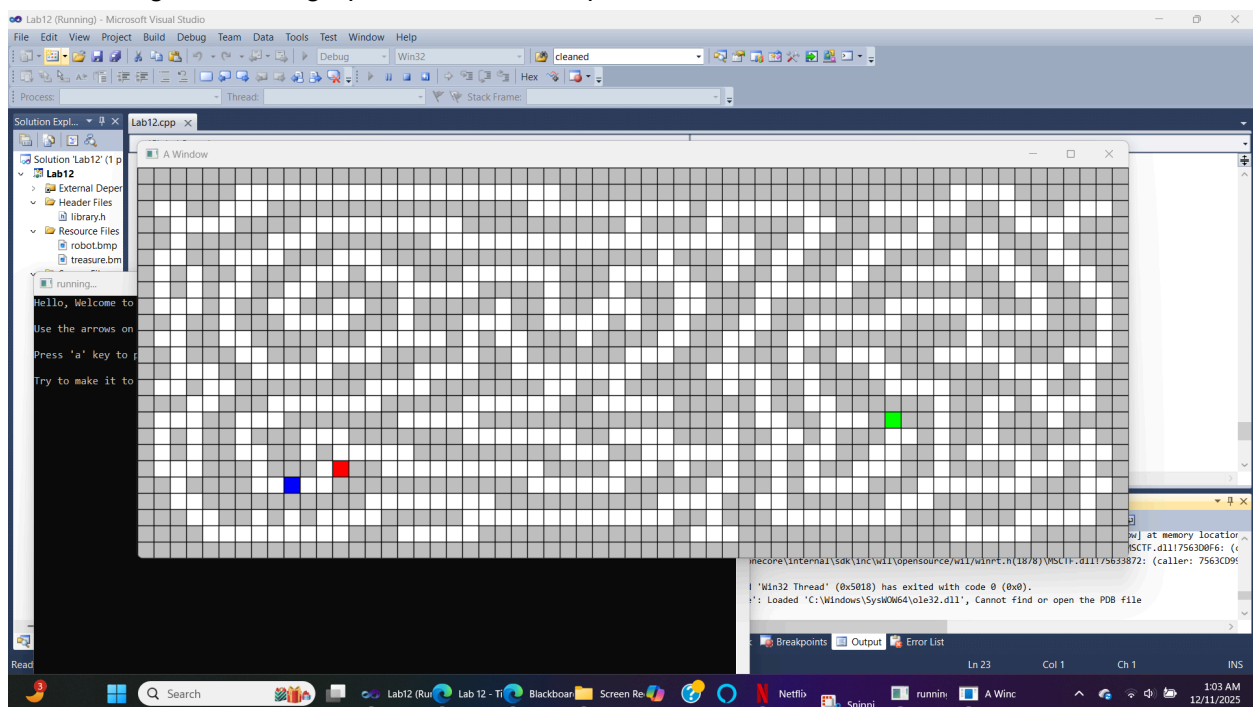
    filled_square(robot_column * cell, robot_row * cell, cell, 2);

    // did u win?
    if (maze[robot_row][robot_column] == '$')
    {
        make_window(700,250);
        set_pen_color(color::white);
        fill_rectangle(0, 0, 700, 250);
        set_pen_color(color::green);
        set_font_size(40);
        move_to(100, 150);
        write_string("YOU FOUND THE TREASURE!!");
    }
}
}
}

```

// note: the robot square (blue) is overlapping with the starting square (red)

This is all i got...cause graphics wont show up...



8. Retracing Steps

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

// THE PART THATS GIVING ME HELL!!!!!!!!!!!!

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else
    {
        set_pen_color(color::white);
    }
}
```

```

    }

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);

    if (which == 1)
    {
        image* treasure_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual
Studio 2010/Projects/Lab12/Lab12/treasure.bmp");
        draw_image(treasure_icon, x, y);
    }
    else if (which == 2)
    {
        image* robot_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual Studio
2010/Projects/Lab12/Lab12/robot.bmp");
        draw_image(robot_icon, x, y);
    }
}

```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'A' key to put the robot in automatic mode." <<
endl << endl << "Press 'B' to backstep to the previous position." << endl << endl << "Try to
make it to the treasure :) (green sqaure)" << endl;
}

```

```

char maze[size][size];
int rows = 0;
int columns = 0;
int start_row = -1, start_column = -1;
int treasure_row = -1, treasure_column = -1;

ifstream fin("maze2a.txt");
if (!fin)
{
    cout << "Can't open file :(";
}

string line;

while (getline(fin, line))
{
    columns = (int)line.length();

    for (int c = 0; c < columns; c++)
    {
        maze[rows][c] = line[c];

        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}

fin.close();

make_window(columns * cell, rows * cell);
set_pen_width(1);

for (int r = 0; r < rows; r++)

```

```

{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell;
        int y = r * cell;

        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {
            moving_space(x, y, cell);
        }
    }
}

```

```

filled_square(start_column * cell, start_row * cell, cell, 2);

```

```

int robot_row = start_row;
int robot_column = start_column;

```

```

bool automatic = false;
const int moves = 11;

```

```

const int max_history = 10000;
int history_row[max_history];
int history_column[max_history];
int history_size = 0;

```

```

while (true)
{
    char c = 0;

```

```

if (!automatic)
{
    c = wait_for_key_typed();
}
else
{
    for (int n = 0; n < moves; n++)
    {
        int r = rand() % 4;
        if (r == 0)
        {
            c = -91;
        }
        if (r == 1)
        {
            c = -89;
        }
        if (r == 2)
        {
            c = -90;
        }
        if (r == 3)
        {
            c = -88;
        }

        int new_row = robot_row;
        int new_column = robot_column;

        if (c == -91)
        {
            new_column--;
        }
        if (c == -89)
        {
            new_column++;
        }
        if (c == -90)
        {
            new_row--;
        }
        if (c == -88)
        {
            new_row++;
        }
    }
}

```

```

    }

    if (maze[new_row][new_column] != 'X')
    {
        if (maze[robot_row][robot_column] == '+')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
        else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
        else
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 3);
        }
        if (history_size < max_history)
        {
            history_row[history_size] = robot_row;
            history_column[history_size] = robot_column;
            history_size++;
        }

        robot_row = new_row;
        robot_column = new_column;
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }
    else
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }

    wait(0.15);
}

char k = wait_for_key_typed();

if (k == 'q')
{
    break;
}
if (k == 'a')
{
    automatic = false;
}

```

```

        continue;
    }
else
    {
        automatic = true;
        continue;
    }
}

if (c == 'q')
{
    break;
}

if (c == 'a')
{
    automatic = true;
    continue;
}

if (c == 'B' || c == 'b')
{
    if (history_size > 0)
    {
        if (maze[robot_row][robot_column] == '+')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
        else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
        else
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 3);
        }
    }

    history_size--;

    int prev_row = history_row[history_size];
    int prev_col = history_column[history_size];

    robot_row = prev_row;

```

```

        robot_column = prev_col;

        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }

    else
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }

    continue;
}

```

```

int new_row = robot_row;
int new_column = robot_column;

if (c == -91)
{
    new_column--;
}
if (c == -89)
{
    new_column++;
}
if (c == -90)
{
    new_row--;
}
if (c == -88)
{
    new_row++;
}
if (maze[new_row][new_column] == 'X')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 2);
    continue;
}

```

```

if (maze[robot_row][robot_column] == '+')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}

```

```

else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}

if (history_size < max_history)
{
    history_row[history_size] = robot_row;
    history_column[history_size] = robot_column;
    history_size++;
}

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

// did u win? :)
if (maze[robot_row][robot_column] == '$')
{
    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::green);
    set_font_size(40);
    move_to(80, 150);
    write_string("YOU FOUND THE TREASURE!!");
}
}
}

// note: the robot square (blue) is overlapping with the starting square (red)

```

[Link to screen recording of working 'B' \(backstep\) key](#)

9. Semi-intelligent Behaviour

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

// THE PART THATS GIVING ME HELL!!!!!!!!!!!!

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else
    {
        set_pen_color(color::white);
    }
}
```

```

    }

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);

    if (which == 1)
    {
        image* treasure_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual
Studio 2010/Projects/Lab12/Lab12/treasure.bmp");
        draw_image(treasure_icon, x, y);
    }
    else if (which == 2)
    {
        image* robot_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual Studio
2010/Projects/Lab12/Lab12/robot.bmp");
        draw_image(robot_icon, x, y);
    }
}

```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'A' key to put the robot in automatic solver

```

```
mode." << endl << endl << "Press 'B' to backstep to the previous position." << endl << endl <<
"Try to make it to the treasure :) (green sqaure)" << endl;
```

```
    char maze[size][size];
    bool visited[size][size];
    char came_from[size][size];
```

```
int rows = 0, columns = 0;
int start_row = -1, start_column = -1;
int treasure_row = -1, treasure_column = -1;
```

```
ifstream fin("maze2a.txt");
```

```
if (!fin)
{
    cout << "Can't open file :(" << endl;
}
```

```
string line;
```

```
while (getline(fin, line))
{
    columns = line.length();
    for (int c = 0; c < columns; c++)
    {
        maze[rows][c] = line[c];
        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}
```

```
fin.close();
```

```
make_window(columns * cell, rows * cell);
set_pen_width(1);
```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell, y = r * cell;
        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {
            moving_space(x, y, cell);
        }
    }
}

```

```

filled_square(start_column * cell, start_row * cell, cell, 2);

```

```

int robot_row = start_row, robot_column = start_column;

```

```

const int max_history = 10000;
int history_row[max_history], history_col[max_history];
int history_size = 0;

```

```

bool auto_on = false;
bool auto_find = false;

```

```

string auto_path =
"LDLLLLUULUURRURRRRRURRRURRRUUURDRDRDRRRRRRRURRRURUUUULUURRRU
URRRRRRRDDLDDRRRRURURURRRUUURURRRDRDRDRDRDRDRDDDDLDDDLLLUUL
LULLUUULLLLLDLLLLLDLDDRDRDRDDLDDRURRUUU";

```

```

string cleaned_path = "";
for (int n = 0; n < (int)auto_path.length(); n++)
{
    char ch = auto_path[n];
    if (ch != ' ')
    {
        cleaned_path +=ch;
    }
}

```

```

int auto_solver = 0;

```

```

for (int r = 0; r < rows; r++)
    for (int c = 0; c < columns; c++)
        visited[r][c] = false;

```

```

visited[robot_row][robot_column] = true;

```

```

while (true)
{
    if (!auto_on)
    {
        char key = wait_for_key_typed();
        if (key == 'q')
        {
            break;
        }
    }

```

```

    if (key == 'A' || key == 'a')
    {
        auto_on = true;
        auto_find = false;
        auto_solver = 0;
        continue;
    }

```

```

    if (key == 'B' || key == 'b')
    {
        if (history_size > 0)
        {

```

```

        if (maze[robot_row][robot_column] == '+')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
        else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
        else
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 3);
        }

        history_size--;
        robot_row = history_row[history_size];
        robot_column = history_col[history_size];

        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }

    continue;
}

```

```

int new_row = robot_row, new_column = robot_column;

```

```

if (key == -91)
{
    new_column--;
}
if (key == -89)
{
    new_column++;
}
if (key == -90)
{
    new_row--;
}
if (key == -88)
{
    new_row++;
}

if (maze[new_row][new_column] == 'X')

```

```

        {
            continue;
        }

if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
else
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 3);
    }

if (history_size < max_history)
    {
        history_row[history_size] = robot_row;
        history_col[history_size] = robot_column;
        history_size++;
    }

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

if (maze[robot_row][robot_column] == '$')
    {
        make_window(700, 250);
        set_pen_color(color::white);
        fill_rectangle(0, 0, 700, 250);
        set_pen_color(color::green);
        set_font_size(40);
        move_to(80, 150);
        write_string("YOU FOUND THE TREASURE!!!");
    }

    continue;
}

```

```

if (auto_find || auto_solver >= cleaned_path.size())
{
    auto_on = false;
    continue;
}

char step = cleaned_path[auto_solver];
auto_solver++;

int new_row = robot_row, new_column = robot_column;

if (step == 'L')
{
    new_column--;
}
if (step == 'R')
{
    new_column++;
}
if (step == 'U')
{
    new_row--;
}
if (step == 'D')
{
    new_row++;
}

if (maze[robot_row][robot_column] == '+')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}
else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}

```

```

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

if (maze[robot_row][robot_column] == '$')
{
    auto_find = true;

    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::green);
    set_font_size(40);
    move_to(80, 150);
    write_string("YOU FOUND THE TREASURE!!");

    auto_on = false;
}

wait(0.1);
}
}

// note: the robot square (blue) is overlapping with the starting square (red)

```

[Link to screen recording of working automatic solver](#)

10. The Yellow Brick Road

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

// THE PART THATS GIVING ME HELL!!!!!!!!!!!!

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else
    {
        set_pen_color(color::white);
    }
}
```

```

    }

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);

    if (which == 1)
    {
        image* treasure_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual
Studio 2010/Projects/Lab12/Lab12/treasure.bmp");
        draw_image(treasure_icon, x, y);
    }
    else if (which == 2)
    {
        image* robot_icon = image_from_file("C:/Users/ajayi/OneDrive/Documents/Visual Studio
2010/Projects/Lab12/Lab12/robot.bmp");
        draw_image(robot_icon, x, y);
    }
}

```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'A' key to put the robot in automatic solver

```

```
mode." << endl << endl << "Press 'B' to backstep to the previous position." << endl << endl <<
"Try to make it to the treasure :) (green sqaure)" << endl;
```

```
    char maze[size][size];
    bool visited[size][size];
    char came_from[size][size];
```

```
int rows = 0, columns = 0;
int start_row = -1, start_column = -1;
int treasure_row = -1, treasure_column = -1;
```

```
ifstream fin("maze2a.txt");
```

```
if (!fin)
{
    cout << "Can't open file :(" << endl;
}
```

```
string line;
```

```
while (getline(fin, line))
{
    columns = line.length();
    for (int c = 0; c < columns; c++)
    {
        maze[rows][c] = line[c];
        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}
```

```
fin.close();
```

```
make_window(columns * cell, rows * cell);
set_pen_width(1);
```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell, y = r * cell;
        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {
            moving_space(x, y, cell);
        }
    }
}

```

```

filled_square(start_column * cell, start_row * cell, cell, 2);

```

```

int robot_row = start_row, robot_column = start_column;

```

```

const int max_history = 10000;
int history_row[max_history], history_col[max_history];
int history_size = 0;

```

```

bool auto_on = false;
bool auto_find = false;

```

```

string auto_path =
"LDLLLLUULUURRURRRRRURRRURRRUUURDRDRDRRRRRRRURRRURUUUULUURRRU
URRRRRRRDDLDDRRRRURURURRRUUURURRRDRDRDRDRDRDRDDDDLDDDLLUUL
LULLUUULLLLLDLLLLLDLDDRDRDRDDLDDRURRUUU";

```

```

string clear_path = "";
for (int n = 0; n < (int)auto_path.length(); n++)
{
    char ch = auto_path[n];
    if (ch != ' ')
    {
        clear_path +=ch;
    }
}

```

```

int auto_solver = 0;

```

```

for (int r = 0; r < rows; r++)
    for (int c = 0; c < columns; c++)
        visited[r][c] = false;

```

```

visited[robot_row][robot_column] = true;

```

```

while (true)
{
    if (!auto_on)
    {
        char key = wait_for_key_typed();
        if (key == 'q')
        {
            break;
        }
    }

```

```

    if (key == 'A' || key == 'a')
    {
        auto_on = true;
        auto_find = false;
        auto_solver = 0;
        continue;
    }

```

```

    if (key == 'B' || key == 'b')
    {
        if (history_size > 0)
        {

```

```

        if (maze[robot_row][robot_column] == '+')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
        else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
        else
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 3);
        }

        history_size--;
        robot_row = history_row[history_size];
        robot_column = history_col[history_size];

        filled_square(robot_column * cell, robot_row * cell, cell, 2);
    }

    continue;
}

```

```

int new_row = robot_row, new_column = robot_column;

```

```

if (key == -91)
{
    new_column--;
}
if (key == -89)
{
    new_column++;
}
if (key == -90)
{
    new_row--;
}
if (key == -88)
{
    new_row++;
}

if (maze[new_row][new_column] == 'X')

```

```

        {
            continue;
        }

if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
else
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 3);
    }

if (history_size < max_history)
    {
        history_row[history_size] = robot_row;
        history_col[history_size] = robot_column;
        history_size++;
    }

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

if (maze[robot_row][robot_column] == '$')
    {
        make_window(700, 250);
        set_pen_color(color::white);
        fill_rectangle(0, 0, 700, 250);
        set_pen_color(color::green);
        set_font_size(40);
        move_to(80, 150);
        write_string("YOU FOUND THE TREASURE!!!");
    }

    continue;
}

```

```

if (auto_find || auto_solver >= clear_path.size())
{
    auto_on = false;
    continue;
}

char step = clear_path[auto_solver];
auto_solver++;

int new_row = robot_row, new_column = robot_column;

if (step == 'L')
{
    new_column--;
}
if (step == 'R')
{
    new_column++;
}
if (step == 'U')
{
    new_row--;
}
if (step == 'D')
{
    new_row++;
}

if (maze[robot_row][robot_column] == '+')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}
else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}

```

```
robot_row = new_row;  
robot_column = new_column;
```

```
filled_square(robot_column * cell, robot_row * cell, cell, 2);
```

```
if (maze[robot_row][robot_column] == '$')  
{  
    auto_find = true;
```

```
    for (int n = 0; n < auto_solver; n++)  
    {  
        int r = start_row;  
        int c = start_column;
```

```
        set_pen_color(color::yellow);
```

```
        for (int i = 0; i < n; i++)  
        {  
            char s = clear_path[i];
```

```
            if (s == 'L')  
            {  
                c--;  
            }  
            if (s == 'R')  
            {  
                c++;  
            }  
            if (s == 'U')  
            {  
                r--;  
            }  
            if (s == 'D')  
            {  
                r++;  
            }  
        }  
    }
```

```
        if (r == start_row && c == start_column)  
        {
```

```

        continue;
    }

    fill_rectangle(c * cell, r * cell, cell, cell);
    square_outline(c * cell, r * cell, cell);
}

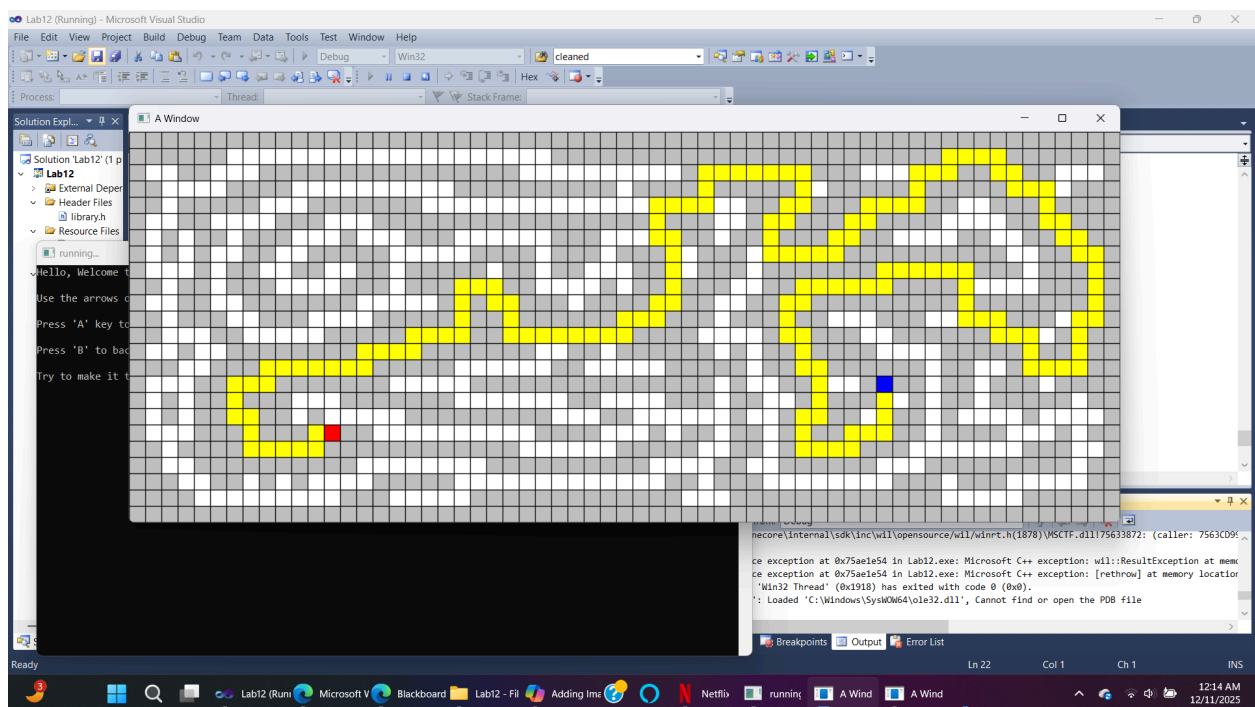
make_window(700, 250);
set_pen_color(color::white);
fill_rectangle(0, 0, 700, 250);
set_pen_color(color::green);
set_font_size(40);
move_to(80, 150);
write_string("YOU FOUND THE TREASURE!!");

auto_on = false;
}

wait(0.1);
}
}

// note: the robot square (blue) is overlapping with the starting square (red)

```



11. Monsters

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;

const int size = 80;
const int cell = 20;

void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}

void monster(int x, int y, int s)
{
    // body
    set_pen_color(color::magenta);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);

    // eyes
    set_pen_color(color::white);
    fill_rectangle(x + s/4, y + s/4, s/6, s/6);
    fill_rectangle(x + (s*2)/3, y + s/4, s/6, s/6);
    set_pen_color(color::black);
    fill_rectangle(x + s/4 + 2, y + s/4 + 2, s/12, s/12);
    fill_rectangle(x + (s*2)/3 + 2, y + s/4 + 2, s/12, s/12);

    // mouth
    set_pen_color(color::black);
    fill_rectangle(x + s/4, y + (s*2)/3, s/2, s/6);
}
```

```

void move_monster(int &monster_row, int &monster_column, int robot_row, int robot_column,
char maze[][size], int rows, int columns)
{
    int prev_row = monster_row;
    int prev_column = monster_column;

    int direction_row = robot_row - monster_row;
    int direction_column = robot_column - monster_column;

    bool vertical = (abs(direction_row) >= abs(direction_column));

    auto move = [&](int r, int c)
    {
        return (r >= 0 && r < rows && c >= 0 && c < columns && maze[r][c] != 'X');
    };

    if (vertical)
    {
        if (direction_row < 0 && move(monster_row - 1, monster_column))
        {
            monster_row--;
            return;
        }
        if (direction_row > 0 && move(monster_row + 1, monster_column))
        {
            monster_row++;
            return;
        }
        if (direction_column < 0 && move(monster_row, monster_column - 1))
        {
            monster_column--;
            return;
        }
        if (direction_column > 0 && move(monster_row, monster_column + 1))
        {
            monster_column++;
            return;
        }
    }
    else
    {
        if (direction_column < 0 && move(monster_row, monster_column - 1))
        {

```

```

        monster_column--;
        return;
    }
    if (direction_column > 0 && move(monster_row, monster_column + 1))
    {
        monster_column++;
        return;
    }
    if (direction_row < 0 && move(monster_row - 1, monster_column))
    {
        monster_row--;
        return;
    }
    if (direction_row > 0 && move(monster_row + 1, monster_column))
    {
        monster_row++;
        return;
    }
}

vertical = !vertical;

if (monster_row - 1 != prev_row || monster_column != prev_column)
{
    if (move(monster_row - 1, monster_column))
    {
        monster_row--;
        return;
    }
}
if (monster_row + 1 != prev_row || monster_column != prev_column)
{
    if (move(monster_row + 1, monster_column))
    {
        monster_row++;
        return;
    }
}
if (monster_row != prev_row || monster_column - 1 != prev_column)
{
    if (move(monster_row, monster_column - 1))
    {
        monster_column--;
        return;
    }
}

```

```

        }
    }
    if (monster_row != prev_row || monster_column + 1 != prev_column)
    {
        if (move(monster_row, monster_column + 1))
        {
            monster_column++;
            return;
        }
    }
    if (move(prev_row, prev_column))
    {
        monster_row = prev_row;
        monster_column = prev_column;
    }
}

```

// THE PART THATS GIVING ME HELL!!!!!!!!!!!!

```

void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else
    {
        set_pen_color(color::white);
    }

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'A' key to put the robot in automatic solver
mode." << endl << endl << "Press 'B' to backstep to the previous position." << endl << endl <<
"Try to make it to the treasure (green square) and outrun the monster at the same time :)" <<
endl;

```

```

    char maze[size][size];
    bool visited[size][size];
    char came_from[size][size];

```

```

    int rows = 0, columns = 0;
    int start_row = -1, start_column = -1;
    int treasure_row = -1, treasure_column = -1;

```

```

    ifstream fin("maze2a.txt");

```

```

    if (!fin)
    {
        cout << "Can't open file :(" << endl;
    }

```

```

    string line;

```

```

while (getline(fin, line))
{
    columns = line.length();
    for (int c = 0; c < columns; c++)
    {
        maze[rows][c] = line[c];
        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}

```

```

fin.close();

```

```

make_window(columns * cell, rows * cell);
set_pen_width(1);

```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell, y = r * cell;
        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else

```

```

    {
        moving_space(x, y, cell);
    }
}

```

```

filled_square(start_column * cell, start_row * cell, cell, 2);

```

```

int robot_row = start_row, robot_column = start_column;

```

```

int monster_row = -1, monster_column = -1;

```

```

while (true)
{
    monster_row = rand() % rows;
    monster_column = rand() % columns;

    if (maze[monster_row][monster_column] != 'X' &&
        !(monster_row == start_row && monster_column == start_column) &&
        !(monster_row == treasure_row && monster_column == treasure_column) &&
        !(monster_row == robot_row && monster_column == robot_column))
    {
        break;
    }
}

```

```

monster(monster_column * cell, monster_row * cell, cell);

```

```

const int max_history = 10000;
int history_row[max_history], history_col[max_history];
int history_size = 0;

```

```

bool auto_on = false;
bool auto_find = false;

```

```

string auto_path =
"LDLLLLUULUURRURRRRRURRRURRRUUURRDRDDRRRRRRRRURRRURUUUULUURRRU
URRRRRRDDLDLDDRRRRURURURRRUURRURRRDRDRDDRRDRDRDDDDLDDDLLLUUL
LULLUUULLLLLDLLLLLDLDDRRDDRDLDLDDRRURUUU";

```

```

string clear_path = "";

```

```

for (int n = 0; n < (int)auto_path.length(); n++)
{
    char ch = auto_path[n];
    if (ch != ' ')
    {
        clear_path +=ch;
    }
}

```

```

int auto_solver = 0;

```

```

for (int r = 0; r < rows; r++)
    for (int c = 0; c < columns; c++)
        visited[r][c] = false;

```

```

visited[robot_row][robot_column] = true;

```

```

while (true)
{
    if (!auto_on)
    {
        char key = wait_for_key_typed();
        if (key == 'q')
        {
            break;
        }

        if (key == 'A' || key == 'a')
        {
            auto_on = true;
            auto_find = false;
            auto_solver = 0;
            continue;
        }

        if (key == 'B' || key == 'b')
        {
            if (history_size > 0)
            {
                if (maze[robot_row][robot_column] == '+')

```

```

{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}
else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}

history_size--;
robot_row = history_row[history_size];
robot_column = history_col[history_size];

filled_square(robot_column * cell, robot_row * cell, cell, 2);

if (maze[monster_row][monster_column] == '+')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 0);
}
else if (maze[monster_row][monster_column] == '$')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 1);
}
else
{
    moving_space(monster_column * cell, monster_row * cell, cell);
}

move_monster(monster_row, monster_column, robot_row, robot_column, maze,
rows, columns);
monster(monster_column * cell, monster_row * cell, cell);

if (monster_row == robot_row && monster_column == robot_column)
{
    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::red);
    set_font_size(30);
    move_to(50, 150);
    write_string("HAHA THE MONSTER GOT YOU! GAME OVER!");
}

```

```
        break;
    }
}
```

```
    continue;
}
```

```
int new_row = robot_row, new_column = robot_column;
```

```
if (key == -91)
{
    new_column--;
}
if (key == -89)
{
    new_column++;
}
```

```
if (key == -90)
{
    new_row--;
}
```

```
if (key == -88)
{
    new_row++;
}
```

```
if (maze[new_row][new_column] == 'X')
{
    continue;
}
```

```
if (maze[robot_row][robot_column] == '+')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}
else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}
```

```

}

if (history_size < max_history)
{
    history_row[history_size] = robot_row;
    history_col[history_size] = robot_column;
    history_size++;
}

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

if (maze[monster_row][monster_column] == '+')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 0);
}
else if (maze[monster_row][monster_column] == '$')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 1);
}
else
{
    moving_space(monster_column * cell, monster_row * cell, cell);
}

move_monster(monster_row, monster_column, robot_row, robot_column, maze, rows,
columns);
monster(monster_column * cell, monster_row * cell, cell);

if (monster_row == robot_row && monster_column == robot_column)
{
    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::red);
    set_font_size(30);
    move_to(50, 150);
    write_string("HAHA THE MONSTER GOT YOU! GAME OVER!");
    break;
}

```

```

if (maze[robot_row][robot_column] == '$')
{
    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::green);
    set_font_size(40);
    move_to(80, 150);
    write_string("YOU FOUND THE TREASURE!!");
}

continue;
}

```

```

if (auto_find || auto_solver >= clear_path.size())
{
    auto_on = false;
    continue;
}

```

```

char step = clear_path[auto_solver];
auto_solver++;

```

```

int new_row = robot_row, new_column = robot_column;

```

```

if (step == 'L')
{
    new_column--;
}
if (step == 'R')
{
    new_column++;
}
if (step == 'U')
{
    new_row--;
}
if (step == 'D')
{
    new_row++;
}

```

```

if (maze[robot_row][robot_column] == '+')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 0);
}
else if (maze[robot_row][robot_column] == '$')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 1);
}
else
{
    filled_square(robot_column * cell, robot_row * cell, cell, 3);
}

```

```

robot_row = new_row;
robot_column = new_column;

```

```

filled_square(robot_column * cell, robot_row * cell, cell, 2);

```

```

if (maze[monster_row][monster_column] == '+')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 0);
}
else if (maze[monster_row][monster_column] == '$')
{
    filled_square(monster_column * cell, monster_row * cell, cell, 1);
}
else
{
    moving_space(monster_column * cell, monster_row * cell, cell);
}

```

```

move_monster(monster_row, monster_column, robot_row, robot_column, maze, rows,
columns);
monster(monster_column * cell, monster_row * cell, cell);

```

```

if (monster_row == robot_row && monster_column == robot_column)
{
    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::red);
}

```

```

    set_font_size(30);
    move_to(50, 150);
    write_string("HAHA THE MONSTER GOT YOU! GAME OVER!");
    break;
}

if (maze[robot_row][robot_column] == '$')
{
    auto_find = true;

    for (int n = 0; n < auto_solver; n++)
    {
        int r = start_row;
        int c = start_column;

        set_pen_color(color::yellow);

        for (int i = 0; i < n; i++)
        {
            char s = clear_path[i];

            if (s == 'L')
            {
                c--;
            }
            if (s == 'R')
            {
                c++;
            }
            if (s == 'U')
            {
                r--;
            }
            if (s == 'D')
            {
                r++;
            }
        }
    }

    if (r == start_row && c == start_column)
    {
        continue;
    }
}

```

```
        fill_rectangle(c * cell, r * cell, cell, cell);
        square_outline(c * cell, r * cell, cell);
    }

    make_window(700, 250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::green);
    set_font_size(40);
    move_to(80, 150);
    write_string("YOU FOUND THE TREASURE!!");

    auto_on = false;
}

wait(0.1);
}
```

[Link to screen recording of moving monster](#)